

CEN 571/CSE 598/CSE 494 Lab 4

Adaptive Dataflow Design using AMD's AI Engines

Yaotian Liu (ASU ID: 1229652231), Xiangyu Zhou (ASU ID: 1229983081)

Experiment 1 (Kernel Vectorization):

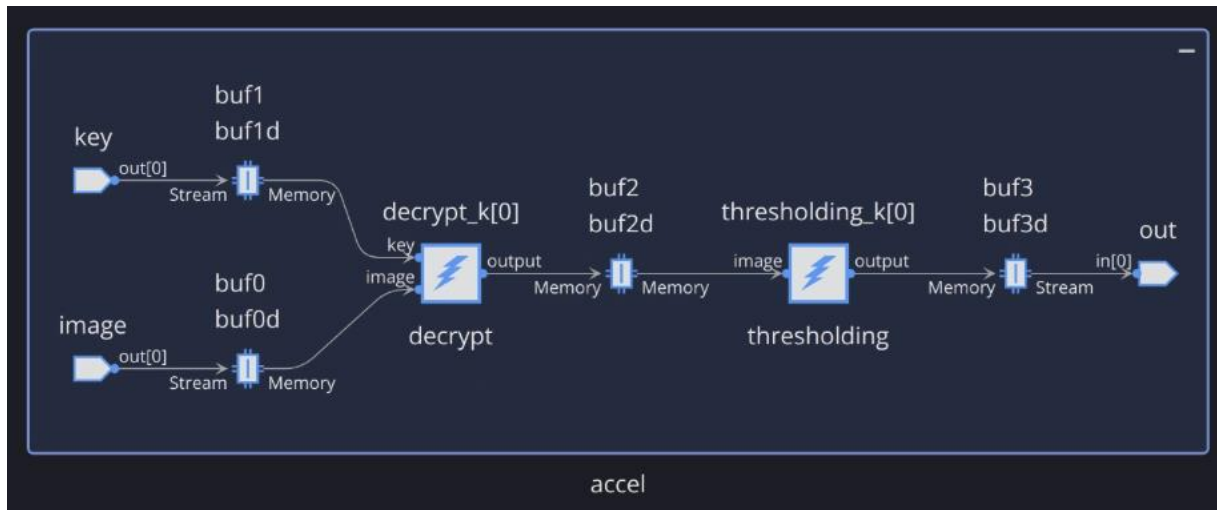
Here are the results for the Kernel vectorization run.

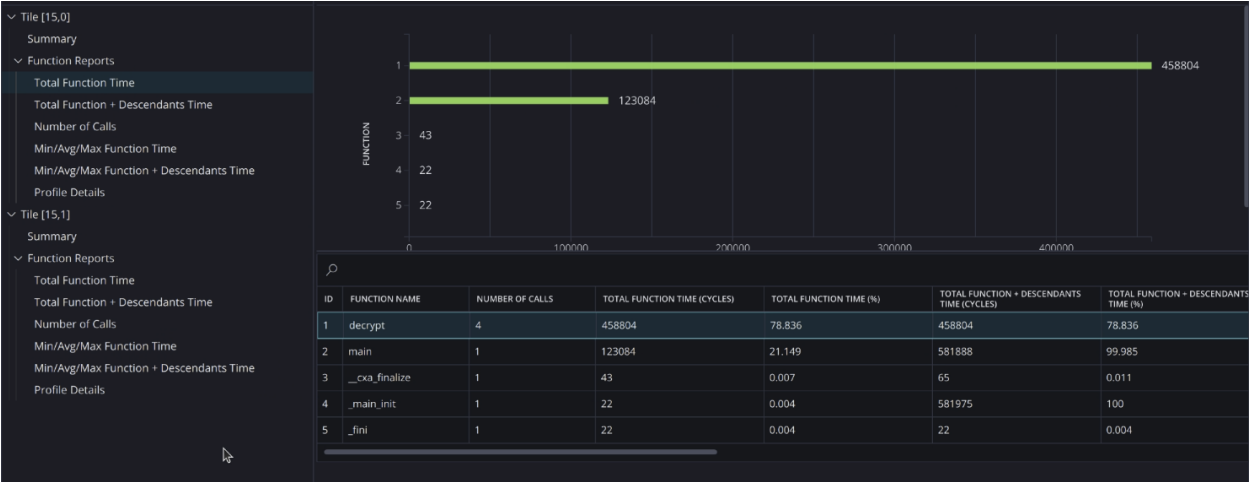
```
Build [x] TASK: AIE SIMULATOR FOR AIE DESIGN [x] PROBLEMS [x]
Vitis Server aie_design:x86sim x aie_design:chw
Guidance: /home/xiangyu/EEE598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim/./Work/reports/guidance.html

INFO: [aiecompiler 77-6440] Use the vitis_analyzer tool to visualize and navigate the relevant reports. Run:
vitis_analyzer /home/xiangyu/EEE598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim/./Work/aie_top_all.aiecompile_summary

Compilation Complete
(WARNING:0, CRITICAL:0, ERROR:0)
INFO: [v++ 60-791] Total elapsed time: 0h 0m 24s
gmake[2]: Leaving directory '/home/xiangyu/EEE598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
[100%] Built target aie_design_libadf.a
gmake[1]: Leaving directory '/home/xiangyu/EEE598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
/Xilinx/Vitis/2024.1/tps/linux64/cmake-3.24.2/bin/cmake -E cmake_progress_start /home/xiangyu/EEE598-Reconfigurable-Computing/lab4/lab4/aie_desi
Build Finished successfully
[4/11/2026, 2:58:44 AM]: Build for aie_design::x86sim with id '053bf687-ae99-4db9-9d72-25ca4cd46fd7' ended.
```

```
Component aie_design [x] TASK: AIE SIMULATOR FOR AIE DESIGN [x] PROBLEMS [x]
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_5.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WRN : -2 : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_5.cm.proc.iss] : Execution tracing should be enabled when requesting function details : chkapi_function_
profiling
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_3.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WRN : -2 : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_3.cm.proc.iss] : Execution tracing should be enabled when requesting function details : chkapi_function_
profiling
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_4.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WRN : -2 : me[::tl.aie.logical.aie_xtln.math_engine.array.tile_15_4.cm.proc.iss] : Execution tracing should be enabled when requesting function details : chkapi_function_
profiling
Info: /OSCI/SystemC: Simulation stopped by user.
IP-INFO: deleting ip PSIP.ps 15
[INFO] : Simulation Finished, Sim result: 0 Total Simulation time 477141200 ps
AIEHLSim feature license is released.
```







Total execution time: 13.603 us

Reconfig lab hic
Reconfig lab hic
Reconfig lab hic
Reconfig lab hic

The following table shows the improvement in terms of the number of cycles and total execution time:

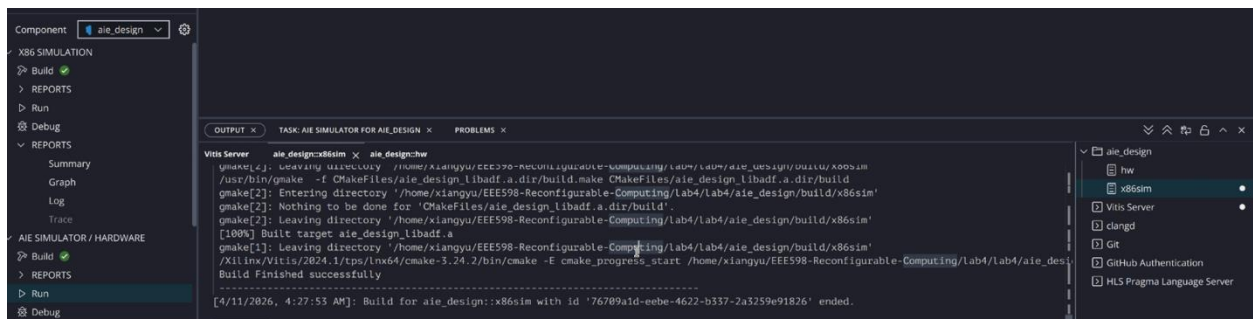
Design	Number of cycles		Total execution
	Decrypt	Thresholding	
Scalar	458804	458804	458.896 us
Vectorization	10304	8256	10us
Speedup factor of Vectorization compares with Scalar	44.5	55.6	46

Provide reasoning for the observed speed up.

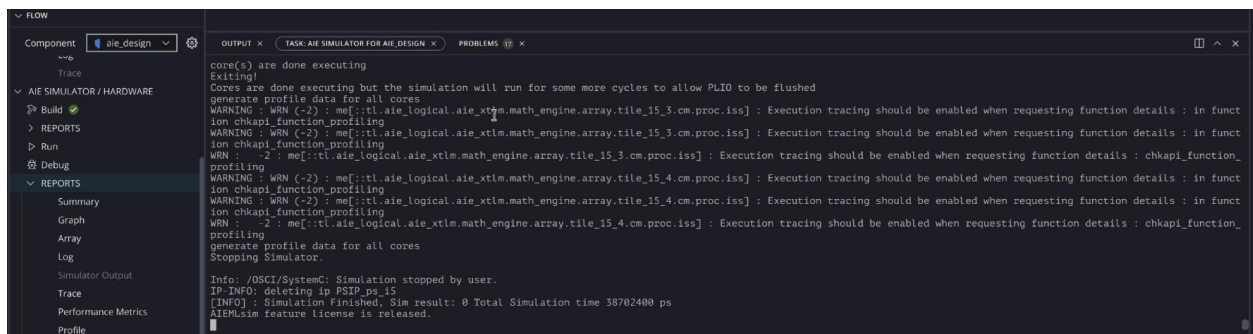
The vectorized implementation achieves approximately 44.5x speedup on the decrypt kernel and 55.6x speedup on the thresholding kernel compared to the scalar baseline. This speedup comes from leveraging the AIE's SIMD (Single Instruction, Multiple Data) vector processing units. The scalar implementation processes one uint8 element per loop iteration, while the vectorized version uses `aie::load_v<32>` to load 32 uint8 elements into a single 256-bit vector register and processes all 32 elements with a single vector instruction (e.g., `aie::bit_xor`, `aie::gt`, `aie::select`). This reduces the total number of loop iterations from 16,384 to 512 (a 32x reduction), and the AIE's vector datapath executes each vector operation in fewer cycles than 32 sequential scalar operations. The combination of reduced loop overhead and efficient hardware-level parallelism within each AIE tile results in the observed speedup.

Experiment 2 (Single buffer vs double buffer (Vector width 32)):

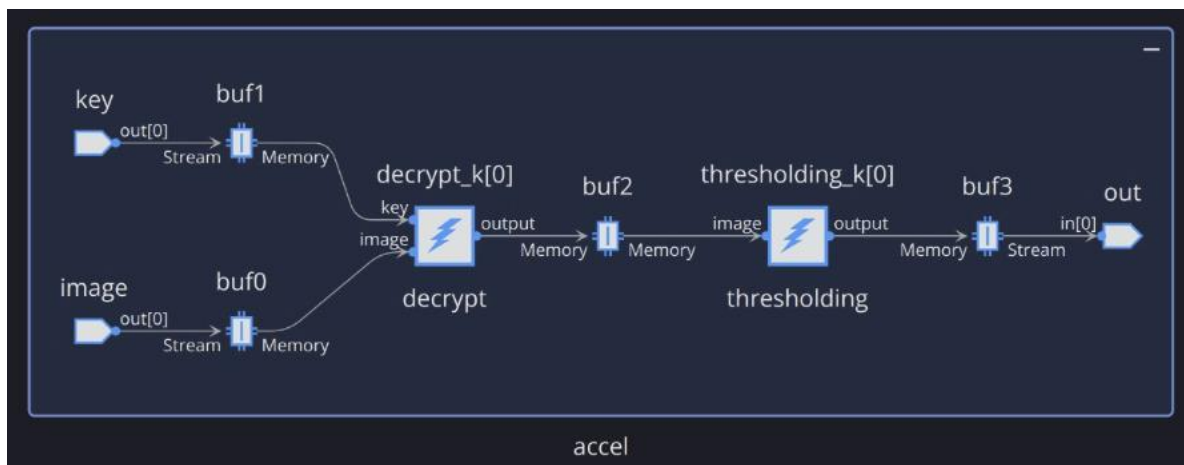
Here are the results for the single buffer design run.

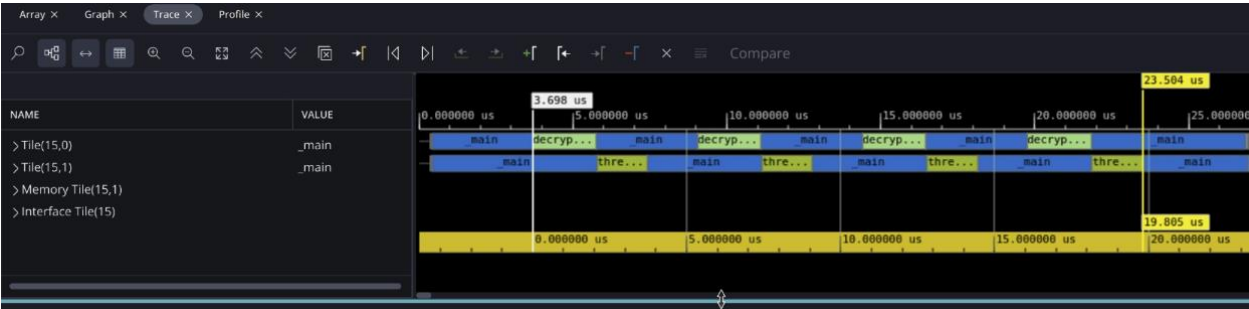
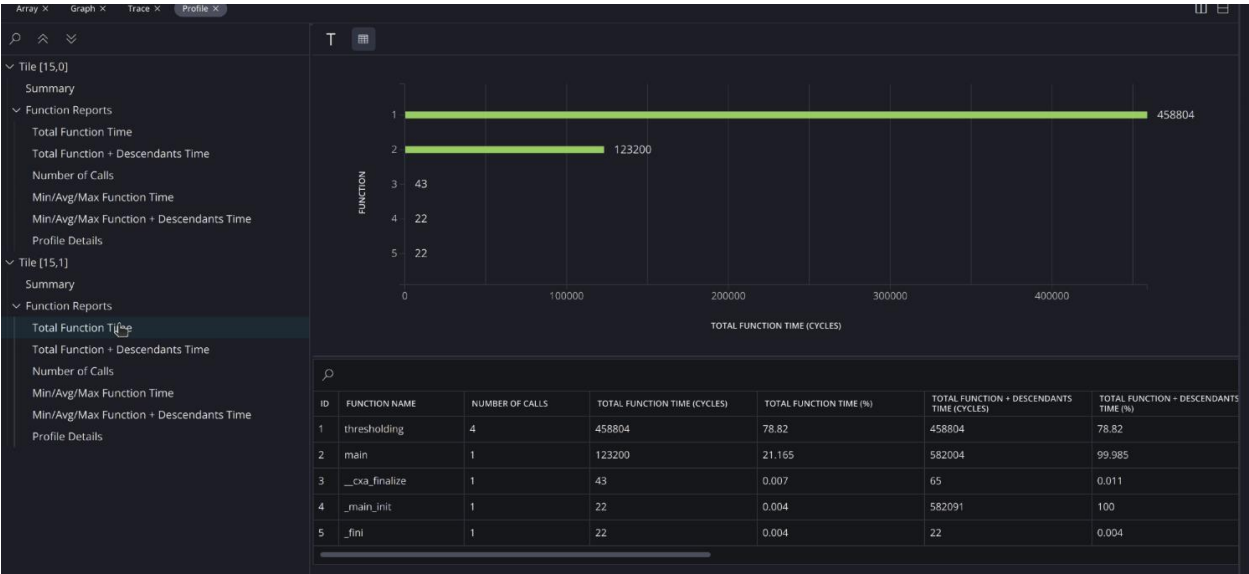
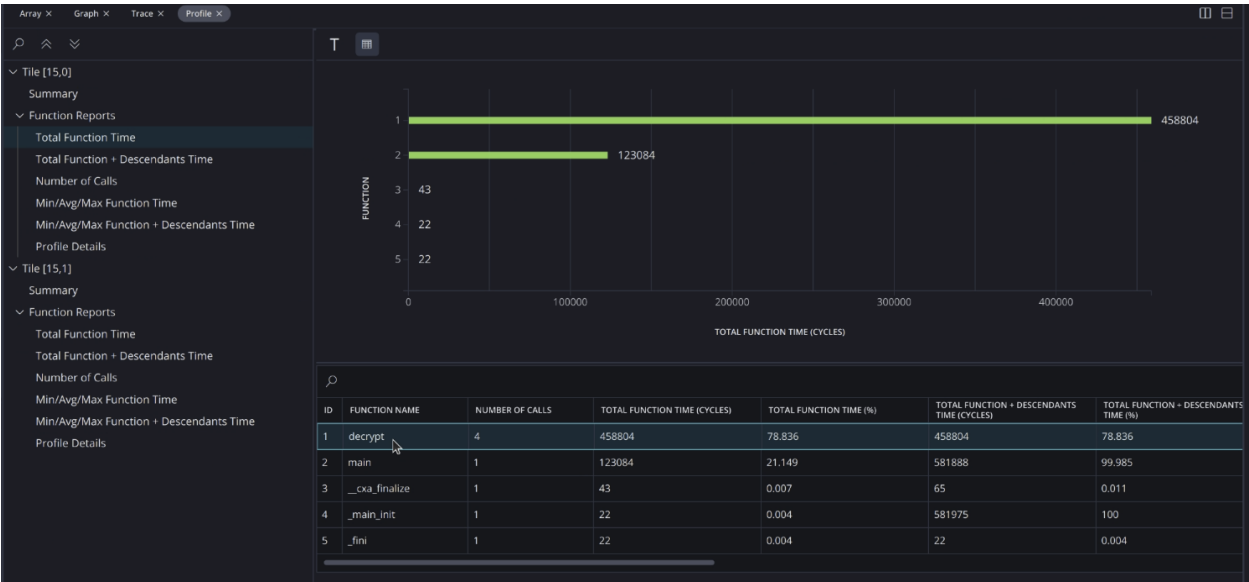


```
gmake[2]: Leaving directory '/home/xiangyu/EEES598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
/usr/bin/gmake -f CMakeFiles/aie_design_libadf.a.dir/build.make CMakeFiles/aie_design_libadf.a.dir/build
gmake[2]: Entering directory '/home/xiangyu/EEES598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
gmake[2]: Nothing to be done for 'CMakeFiles/aie_design_libadf.a.dir/build'.
gmake[2]: Leaving directory '/home/xiangyu/EEES598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
[180s] Built target aie_design_libadf.a
gmake[1]: Leaving directory '/home/xiangyu/EEES598-Reconfigurable-Computing/lab4/lab4/aie_design/build/x86sim'
/Xilinx/Vitis/2024.1/tpls/lnx64/cmake-3.24.2/bin/cmake -E cmake_progress_start /home/xiangyu/EEES598-Reconfigurable-Computing/lab4/lab4/aie_des
Build Finished successfully
-----
[4/11/2026, 4:27:53 AM]: Build for aie_design::x86sim with id '76709a1d-eebe-4622-b337-2a3259e91826' ended.
```

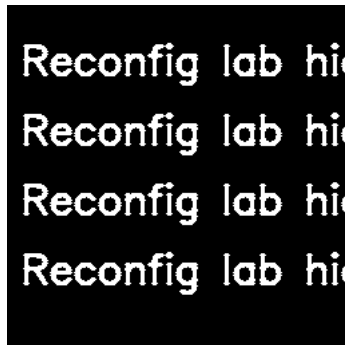


```
core(s) are done executing
Exiting!
Cores are done executing but the simulation will run for some more cycles to allow PLIO to be flushed
generate profile data for all cores
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_3.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_3.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WRN : -2 : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_3.cm.proc.iss] : Execution tracing should be enabled when requesting function details : chkapi_function_
profiling
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_4.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WARNING : WRN (-2) : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_4.cm.proc.iss] : Execution tracing should be enabled when requesting function details : in funct
ion chkapi_function_profiling
WRN : -2 : me[::tl.aie.logical.aie_xtlm.math_engine.array.tile_15_4.cm.proc.iss] : Execution tracing should be enabled when requesting function details : chkapi_function_
profiling
generate profile data for all cores
Stopping Simulator.
Info: /OSCI/SystemC: Simulation stopped by user.
IP-INFO: deleting ip PSIP_ps_15
[INFO] : Simulation Finished, Sim result: 0 Total Simulation time 38702400 ps
AIEMLsim feature license is released.
```





Total execution time: 19.805 us



The following table shows the improvement in terms of the number of cycles and total execution time of the double buffer design compared with the single buffer design:

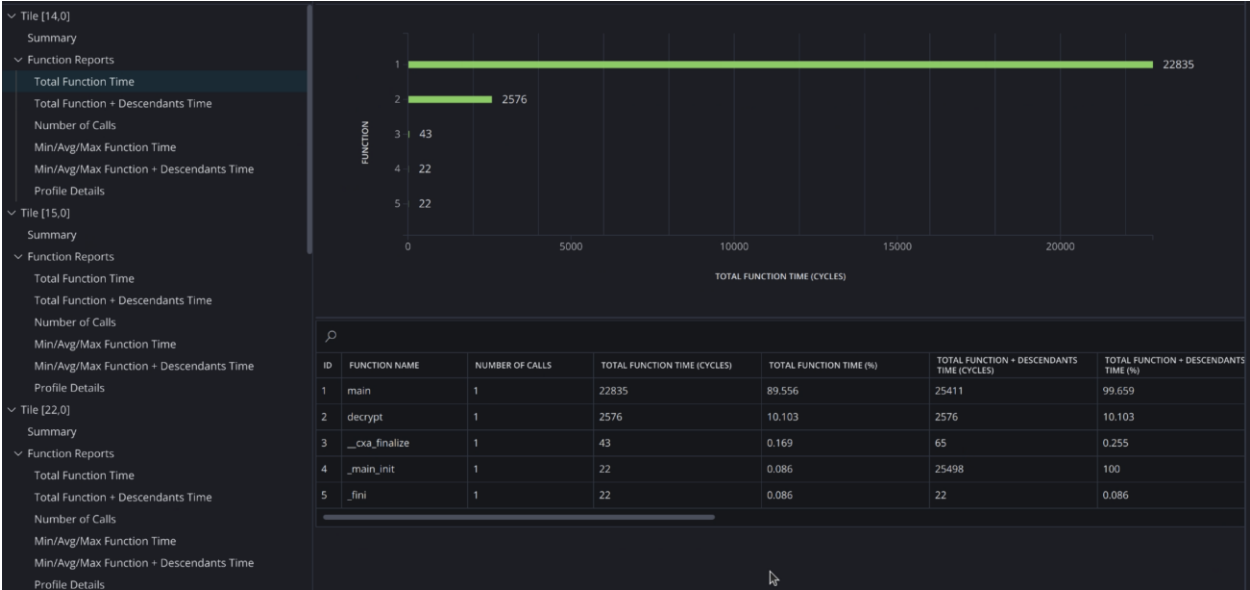
Design	Number of cycles		Total execution
	Decrypt	Thresholding	
Single Buffer	10304	8256	19.8 us
Double Buffer	10304	8256	10 us
Speedup factor of Double buffer compared with Single Buffer	1.0	1.0	2.0

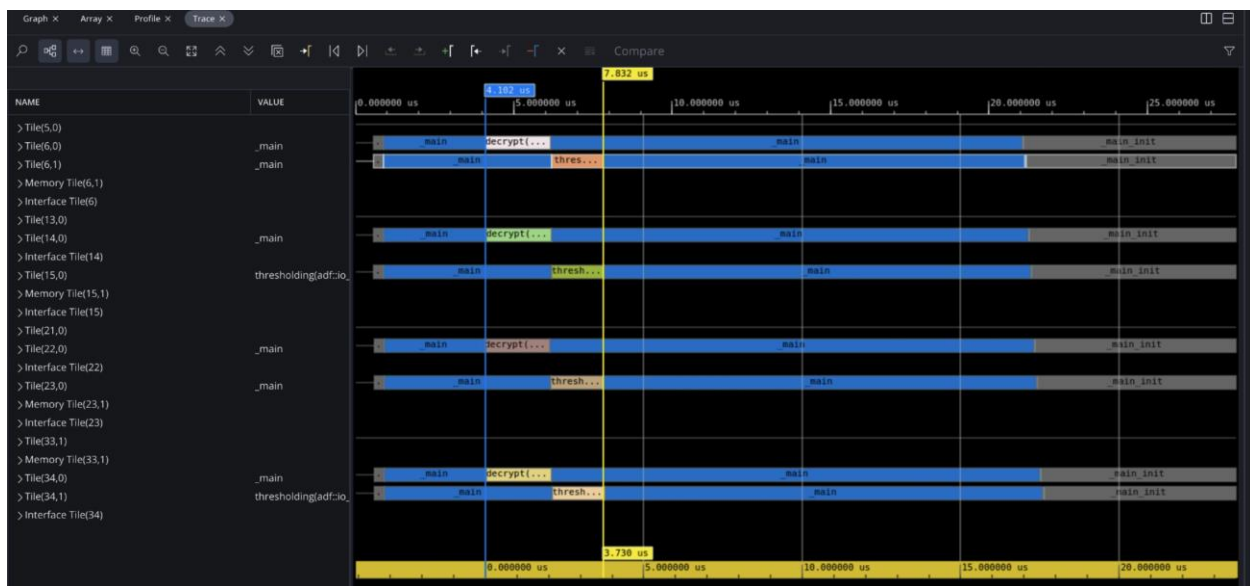
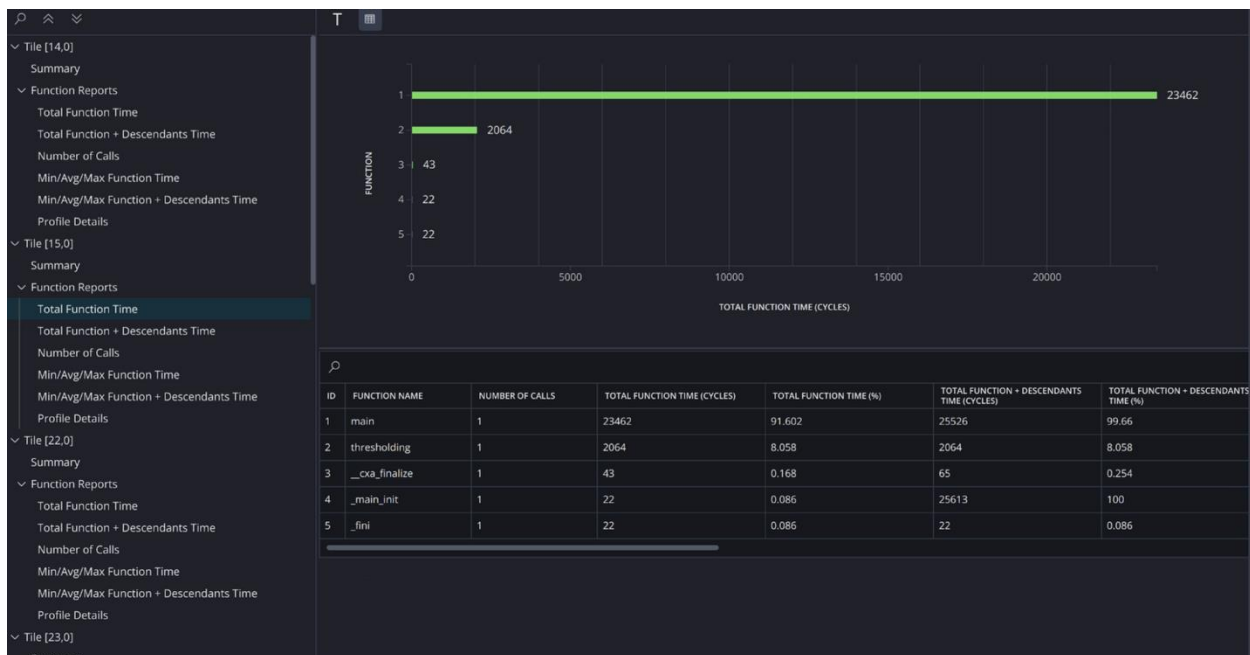
Do you see any slowdown from single-buffering? If yes, where is the slowdown coming from?

Yes, there is a significant slowdown from single-buffering. The total execution time nearly doubles (~19.8 us vs ~10 us) compared to double-buffering, despite the kernel cycle counts being identical (decrypt: 10304, thresholding: 8256). The slowdown comes from the inability to overlap DMA data transfers with kernel computation. With single buffering, the kernel must wait until the entire buffer is written by DMA before it can begin processing, and the next DMA transfer cannot start until the kernel finishes and releases the buffer. With double buffering (ping-pong), one buffer is used by the kernel for computation while the other buffer simultaneously receives the next batch of data via DMA, effectively hiding the data transfer latency behind the computation time.

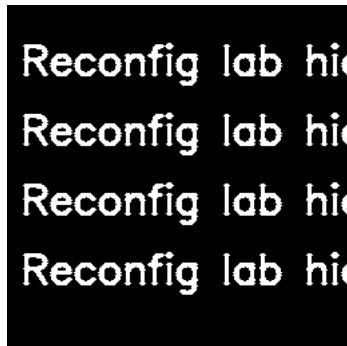
Experiment 3 (Parallelizing your design):

Here are the results for the Parallelizing design run.





Total execution time: 3.730 us



The following table shows the improvement in terms of the number of cycles and total execution time of the Parallelizing design compared with the Vectorization design:

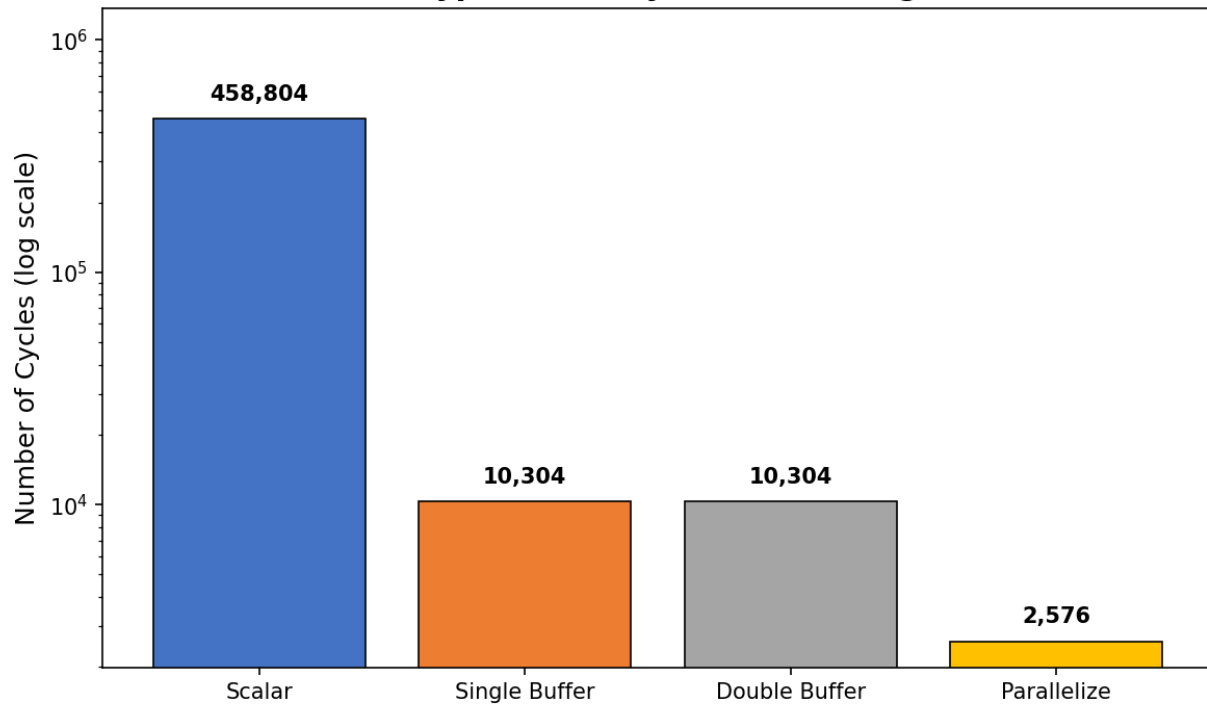
Design	Number of cycles		Total execution
	Decrypt	Thresholding	
Vectorization	10304	8256	10 us
Parallelizing	2576	2064	3.796 us
Speedup factor of Parallelizing compared with Vectorization	4.0	4.0	2.6

Provide reasoning for the observed speed up.

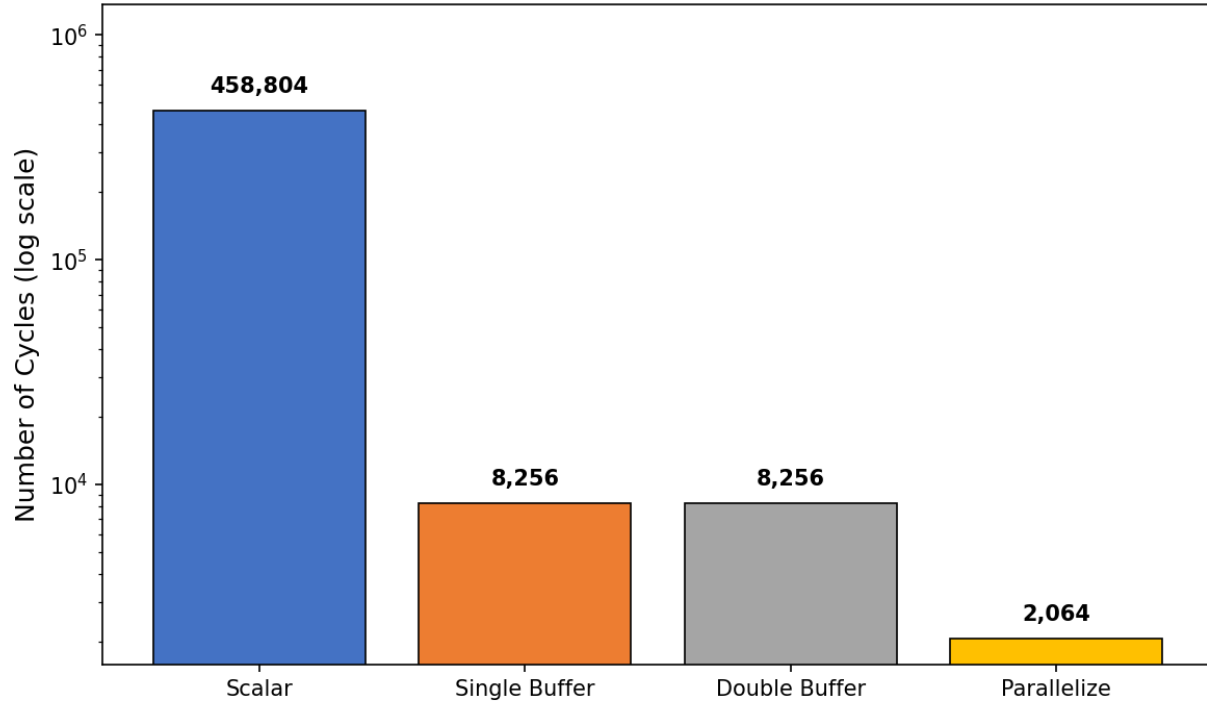
The parallelized design achieves approximately 4x speedup over the vectorized (two-AIE) design. Instead of processing four 128×128 chunks sequentially over four iterations on a single decrypt-thresholding pipeline, the parallel design replicates the pipeline across four separate AIE tile pairs (8 AIEs total). Each pipeline processes one chunk simultaneously in a single iteration. Since all four pipelines execute concurrently, the total execution time is reduced to roughly 1/4 of the sequential vectorized design, achieving near-linear speedup with the number of parallel pipelines.

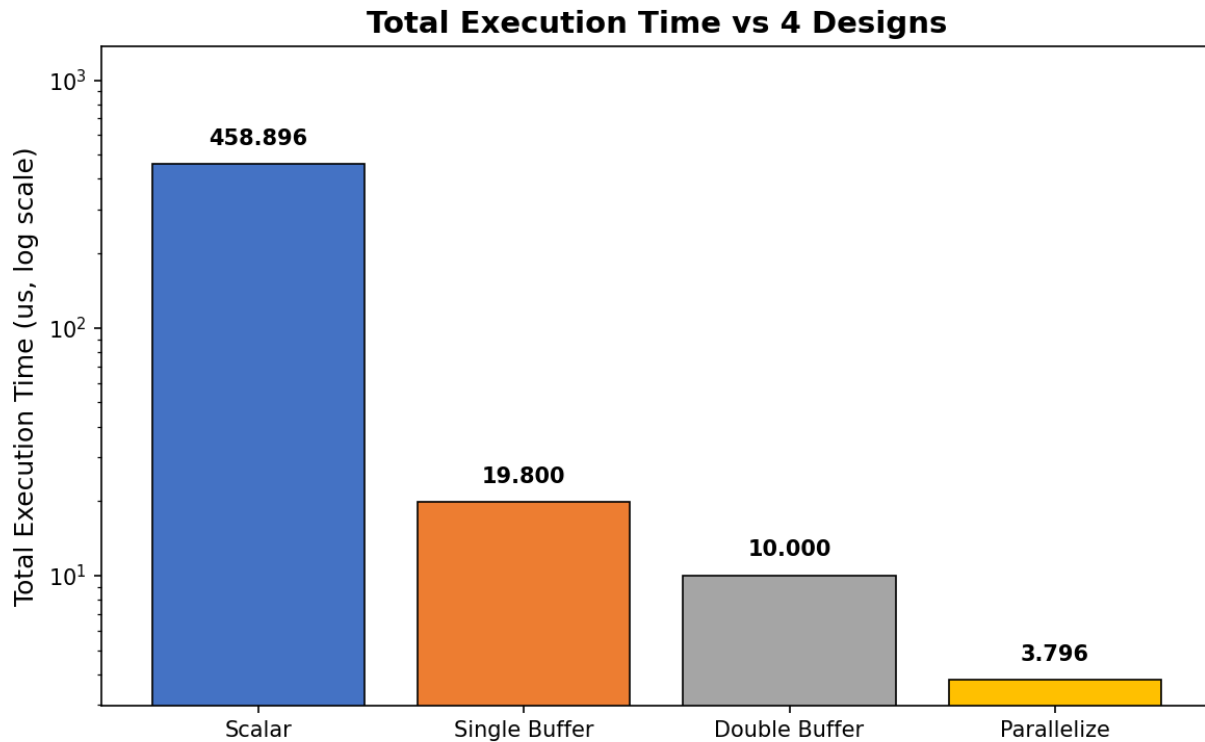
A bar chart for Final Comparison:

Decrypt Kernel Cycles vs 4 Designs



Thresholding Kernel Cycles vs 4 Designs





The following table is a comparison of the final comparison used for drawing the bar chart:

Design	Scalar	Single buffer	Double buffer	Parallelize
Total execution (us)	485.896	19.8	10	3.796
number of cycles of Decrypt	458804	10304	10304	2576
number of cycles of Thresholding	458804	8256	8256	2064

Fine grained pipeline using Cascade <Extra Credit 10 pts>

<Screenshot of the X86 simulation>

< Screenshot of the aiesim flow from the Vitis window>

<Screenshot of the ADF Graph>

<Screenshot of the AIE Array of Cascade >

< Screenshot showing a graph that plots Function against Total Function Time (measured in cycles) for the decrypt kernel in profile>

<Screenshot of the result of decrypt kernel's table in profile>

< Screenshot showing a graph that plots Function against Total Function Time (measured in cycles) for the thresholding kernel in profile>

<Screenshot of the result of the thresholding kernel's table in profile>

<Screenshot of the trace from the report of the Cascade marked start and end time>

Total execution time: end time - start time = Total execution time

<Screenshot of the output decrypted image including message >

Do you see any improvement in performance compared to Single/Double buffer vectorized implementation?